Refactoring and Design Patterns

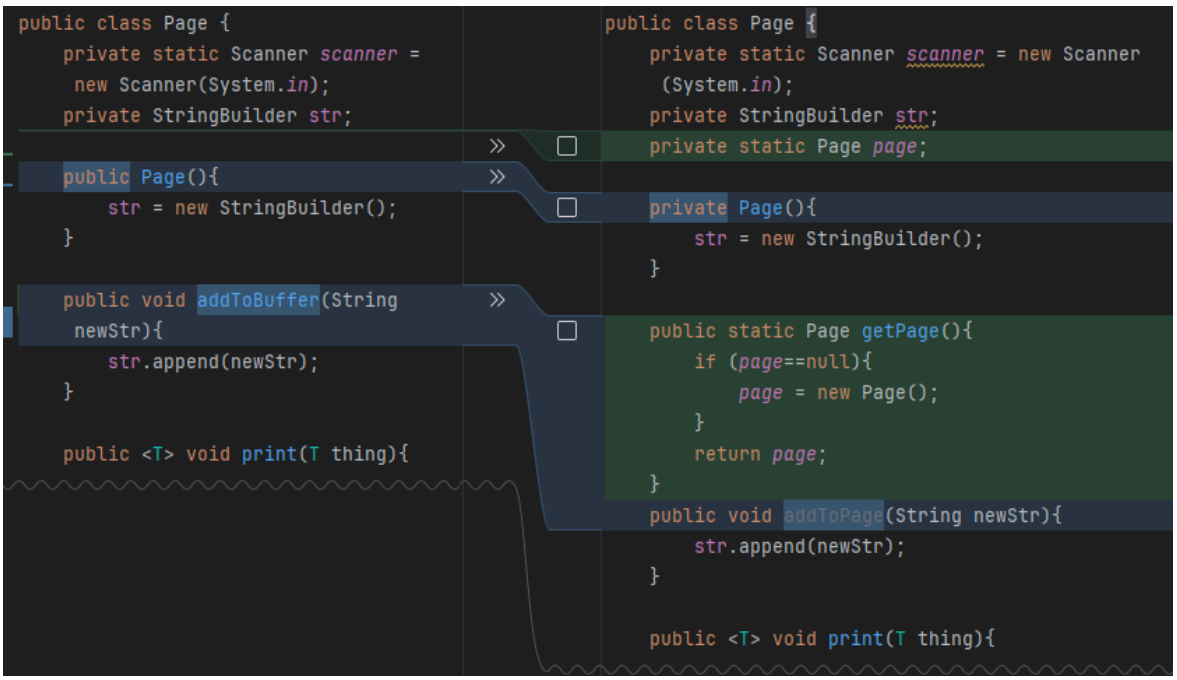
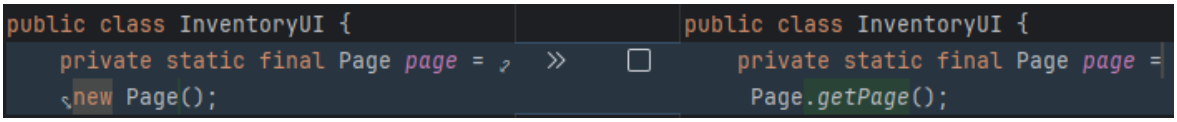
This document was formatted and exported from software called Obsidian with some themes. It is a markdown notetaking application that I really enjoy. This was not written by any sort of AI, I simply enjoy formatting my information neatly.

Design Patterns

1 - Singleton for `Page` class

Implementation

> useful and relevant pattern implemented correctly. Shows a good understanding of the pattern and its principles.

- `Page` Changes
  - Updated `Page` constructor and added `getPage`
  - 
```java
public class Page {
    private static Scanner scanner =
     new Scanner(System.in);
    private StringBuilder str;

    public Page(){
        str = new StringBuilder();
    }

    public void addToBuffer(String
     newStr){
        str.append(newStr);
    }

    public <T> void print(T thing){
```
```java
public class Page {
    private static Scanner scanner = new Scanner
     (System.in);
    private StringBuilder str;
    private static Page page;

    private Page(){
        str = new StringBuilder();
    }

    public static Page getPage(){
        if (page==null){
            page = new Page();
        }
        return page;
    }
    public void addToPage(String newStr){
        str.append(newStr);
    }

    public <T> void print(T thing){
```
- All other `UI` classes:
  - Classes affected: `InventoryUI`, `CampaignUI`, `InventoryUI`, `QuestEventUI`, `RealmUI`, `UserUI`, `WorldTimeUI`
  - Replaced constructor with public `getPage` method.
  - 
```java
public class InventoryUI {
    private static final Page page =
     new Page();
```
```java
public class InventoryUI {
    private static final Page page =
     Page.getPage();
```
    This is repeated for all UI classes.

Motivation / Rationale

> explanation that is sufficiently clear and descriptive, balancing thoroughness and brevity. Demonstrates a good understanding of the refactoring and its principles?

Right now, the UI writes to the terminal as output through a Page class, but in the future we may want to support other forms of output, and perhaps different concurrent threads of users / campaigns at the same time. Because of this, we want the things that the user is sending to the output to be controlled by 1 manager rather than separate objects, especially since they are all going to the same place. This way it can be more tightly coupled / managed and we can control the output more specifically. Also, having only 1 singleton instead of creating various `Page` objects all over is good for performance.

Impact

> How far-reaching is the change to incorporate the pattern, how much it changed the design.

This had very little impact on the design. The dependencies stayed the same, but *all* the classes required changes to how they accessed page. `Page` got an extra method and all the UI classes required 1 line change to access the object through the getter.

2 - Command Pattern for choices

Sources used for reference, since I had to do quite a bit of research on how this worked and could be implemented.: refactor guru , game programming patterns , geeksforgeeks , wikipedia , medium.com ,

Motivation / Rationale

> explanation that is sufficiently clear and descriptive, balancing thoroughness and brevity. Demonstrates a good understanding of the refactoring and its principles?

I thought it would be a good idea to move to a Command Pattern for choices since it will provide another layer of abstraction between the choices and the methods that they run, as well as removing the hard-coded button-to-method strategy that I am using right now. This way in the future, if we ever want to add options, we can simply modify the prompt, and add a new command and set it to trigger the method of choice.
This will help scale the software and may allow the user to configure their own layout of choices in the future, as `command_i` can be set to different options depending on what is wanted.
Also, it breaks apart the `InventoryUI` class into separate components, which helped with the Large Class code smell that was present.

Implementation

> useful and relevant pattern implemented correctly. Shows a good understanding of the pattern and its principles.

For this assignment, I only implemented in 1 UI page, the `InventoryUI` page. I broke the individual options that the page into their separate commands and made the `InventoryUI` choose a command based on user input. Each `command_i` was configured to one of the `InventoryCommands`, which performed the necessary actions.

As a lot of the functions that were run on the Inventory within the `InventoryUI` included user input, we decided to still keep that separate from the `Inventory` class since that be kept entirely separate from user input. In a classical Command structure, the Concrete Commands would typically call 1 method within the Business Logic class (in this case, `Inventory`), but we wanted to, again, keep the UI out of it.

Here are the changes:

- New `Command` Interface in `Core` class.
  -
    ```java
    package Core;

    public abstract interface Command {
        public abstract void execute();
    }
    ```
- New `InventoryCommand` abstract class
  - Moved `printAndQueryInventory` methods into here.
  - Also created `execute` redirect to handle input.
    -
      ```java
      package InventoryCommands;

      import Core.Command;
      import RPGDomain.Character;
      import RPGDomain.Inventory;
      import RPGDomain.Item;
      import ViewsAndTimelines.Page;

      public abstract class InventoryCommand implements Command {  11 usages  4 inheritors
          public static final Page page = Page.getPage();
          public Character character=null;
          public static int printAndQueryInventory(RPGDomain.Character character) {...}
          public static int printAndQueryInventory(Character character, String extraPrompt){...}

          @Override  1 usage  4 overrides
          public void execute() {
              execute(character);
          }
          public void execute(Character c) {  4 usages
              character = c;
              execute();
          }
      }
      ```
- Several Concrete `InventoryCommands` classes:
  - created `AddItemsCommand`
    - Moved `addItems` and `addItem` into here.

```java
package InventoryCommands;

import RPGDomain.Character;
import RPGDomain.Item;

public class AddItemsCommand extends InventoryCommand{  4 usages
    public void execute(){  1 usage
        String name;
        while (true){
            if (addItem(character) == null) break;
        }
    }
    public static Item addItem(Character character){...}
}
```

- created `RemoveItemCommand`
  - moved `removeItem` into here

```java
package InventoryCommands;

public class RemoveItemCommand extends InventoryCommand{  4 usages
    @Override  1 usage
    public void execute(){
        int valid = page.acceptIntUntil( prompt: "Are you sure you want to remove an item?\n1 --- yes\n0 --- no\n",  inclusiveMax: 1);
        if (valid == 0) return;
        int choice = InventoryCommand.printAndQueryInventory(character,  extraPrompt: "to throw away");
        if (choice == -1) return;
        boolean res = character.getInventory().getItems().remove(
                character.getInventory().getItems().get(choice)
        );
        if (res == false) page.print("ERROR: ITEM WAS NOT REMOVED");
    }

}
```

- created `LeaveCommand`

```java
package InventoryCommands;

public class LeaveCommand extends InventoryCommand{  1 usage
    public void execute(){ return; }  1 usage
}
```

- created `SelectInventoryCommand`
  - moved `selectInventory` and `displayItem` methods into `SelectInventoryCommand`

```java
package InventoryCommands;

import RPGDomain.Item;

public class SelectInventoryCommand extends InventoryCommand{ 1 usage
    public void execute(){ 1 usage
        int choice = printAndQueryInventory(character, extraPrompt: "to see more options");
        if (choice == -1) return;
        else displayItem(character.getInventory().getItems().get(choice));
    }
    private static void displayItem(Item item){ 1 usage
        page.print(item.getName() + " --- " + item.getDesc() + " " + item.getRarity() + '\n');
    }
}
```

- `InventoryUI` changes:
  - Added 4 static `Command` objects

```java
public class InventoryUI {
    private static final InventoryCommand command_2 = new RemoveItemCommand();
    private static final InventoryCommand command_1 = new SelectInventoryCommand();
    private static final InventoryCommand command_3 = new AddItemsCommand();
    private static final InventoryCommand command_0 = new LeaveCommand();
```

  - Added `intToCommand` method to return `Command` objects.

```java
    public static InventoryCommand intToCommand(int i){
        switch (i) {
            case 0: return command_0;
            case 1: return command_1;
            case 2: return command_2;
            case 3: return command_3;
        }
        throw new Error("INVALID CHOICE, NO COMMAND BOUND");
    }
```

  - Modified `manageInventory` to use command `execute` method.

```java
    public static void manageInventory(Character character) {

        int choice = -1;
        while (choice != 0) {
            choice = page.acceptIntUntil(prompt, 3);
            choice = page.acceptIntUntil(MANAGE_PROMPT, 3);
            page.nextScreen();
            switch (choice) {
                case 0-> page.print("leaving inventory\n");
                case 1-> selectInventory(character);
                case 2-> removeItem(character);
                case 3-> addItems(character);
            }
        }
    }
    private static int printAndQueryInventory(Character character) {
        return printAndQueryInventory(character, "");
            InventoryCommand command = intToCommand(choice);
            command.execute(character);
        }
```

- Moved methods to `InventoryCommand`s
  - moved `printAndQueryInventory` methods into `InventoryCommand`
  - moved `selectInventory` methods into `SelectInventoryCommand`
  - moved `displayItem` methods into `SelectInventoryCommand`
  - moved `removeItem` methods into `RemoveItemCommand`
  - moved `addItems` methods into `AddItemCommand`
  - moved `addItem` methods into `AddItemCommand`
- `QuestEventUI` changes
  - Added 2 command Items to replace calls to `InventoryUI`

```java
public class QuestEventUI {
    private static final Page page = new Page();
    public static QuestEvent
```

```java
public class QuestEventUI {
    private static final Page page = Page.getPage();
    private static final AddItemsCommand addItemsCommand = new AddItemsCommand();
    private static final RemoveItemCommand removeItemCommand = new RemoveItemCommand();
    public static QuestEvent chooseQuestEvent
```

  - In `rewardCharacter` method

```java
InventoryUI.addItem(character);
```

```java
addItemsCommand.execute(character);
}
private static void punishCharacter(QuestEve
```

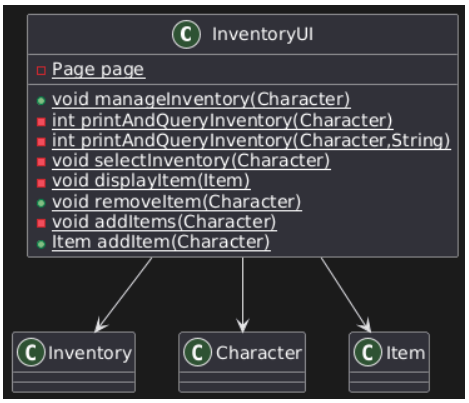
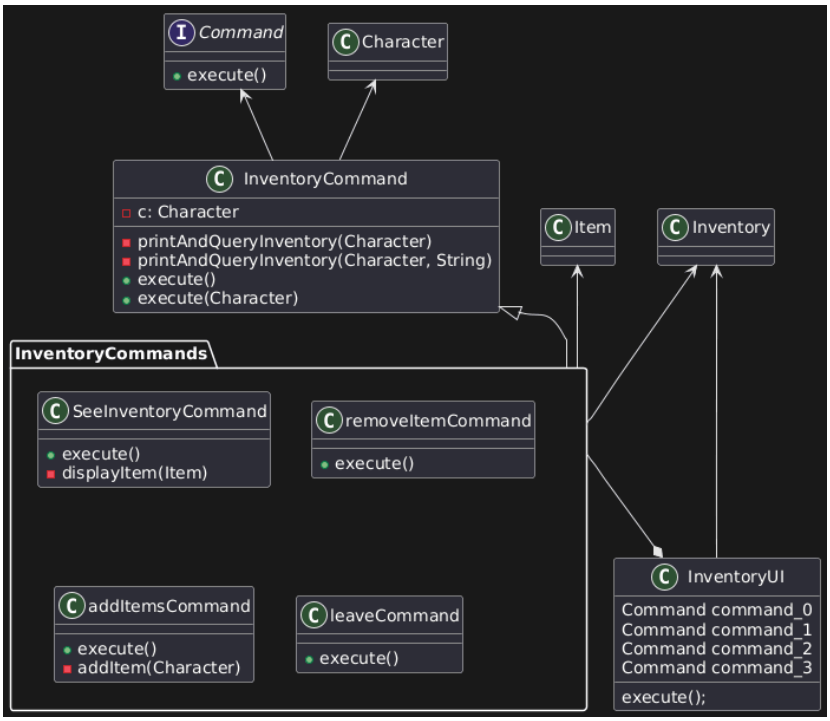
  - In `punishCharacter` method

```java
InventoryUI.removeItem(character);
```

```java
removeItemCommand.execute(character);
}
public static String timelineString(Campaign c
```

Impact

> How far-reaching is the change to incorporate the pattern, how much it changed the design.

Before	After
	

The impact on the design was quite dramatic. We added several more classes but we managed to reroute some dependencies to be absorbed by the individual `Command` classes. There were 2 stray methods within the `QuestEventUI` class that were affected by this, but they were able to depend on the individual `Command`s instead of the entire `InventoryUI` class. However, we did end up adding 5 classes and one interface, which is quite a lot of bloat for the system, but may pay off if there were plans to scale.

Refactors

1 - Removing `User` Dependency within `CampaignUI`.

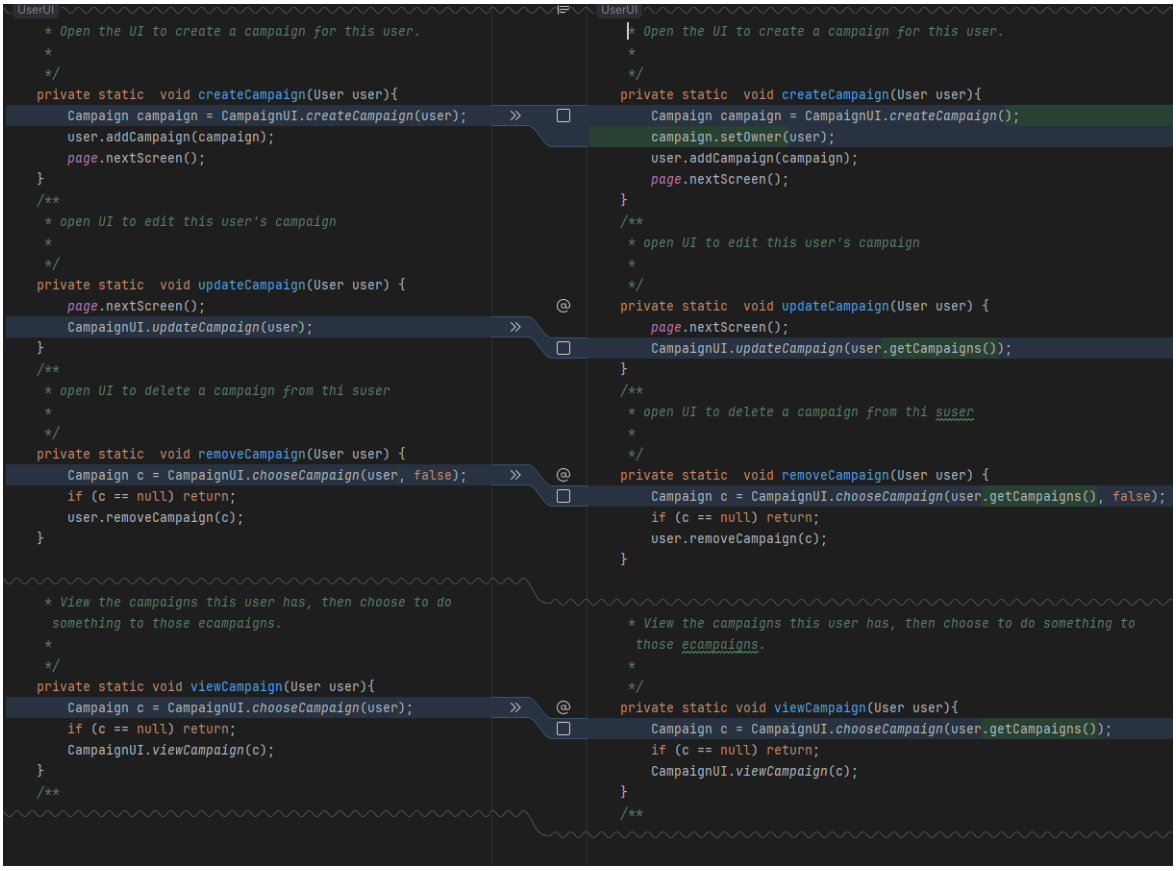
The issue was this: inappropriate intimacy
The refactor was this: Change Function Parameters.

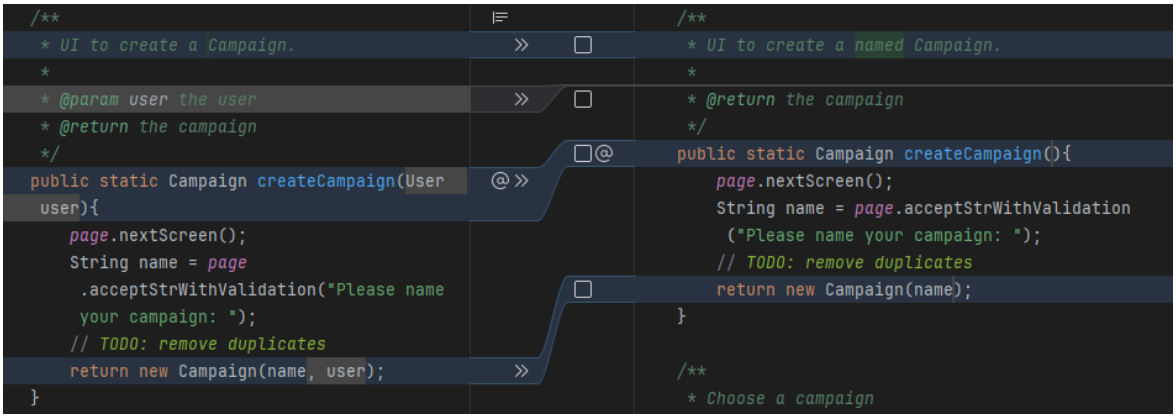
> This was listed as a refactor here.

Implementation

> useful and relevant pattern implemented correctly. Shows a good understanding of the pattern and its principles.

- `UserUI` changes:

- Replaced all `user` param passes with `user.getCampaigns()`
  - 

- `CampaignUI` changes
  - `createCampaign` removes param
    - 

  - `chooseCampaign` substitutes `user.getCampaigns()` with `campaigns`
    -

    -

```java
public static Campaign chooseCampaign(User user,
 boolean includeArchived){
    StringBuilder prompt = new StringBuilder("Please
      choose a Campaign\n");
    Campaign choice = null;
    int i = 1;
    if (user.getCampaigns().isEmpty()){
        page.print("Sorry, there are no campaigns,
         please choose something else\n");
        return null;
    }
    for (Campaign c : user.getCampaigns()){
        prompt.append(i).append(" --- ").append(c
         .getName()).append('\n');
        ++i;
    }
    prompt.append("0 -- exit\n");
    int input = -1;
    while (input == -1){
        input = page.acceptIntUntil(prompt.toString
         (), i-1);
        if ( !includeArchived && input != 0 &&
         user.getCampaigns().get(input-1)
         .getArchived() == true){
            page.nextScreen();
            page.print("Please choose something not
             actively archived");
        } else break;
    }


    page.nextScreen();
    if (input == 0) return null;
    else return user.getCampaigns().get(input-1);
}
```

```java
public static Campaign chooseCampaign(List<Campaign>
 campaigns, boolean includeArchived){
    StringBuilder prompt = new StringBuilder("Please choose
      a Campaign\n");
    Campaign choice = null;
    int i = 1;
    if (campaigns.isEmpty()){
        page.print("Sorry, there are no campaigns, please
         choose something else\n");
        return null;
    }
    for (Campaign c : campaigns){
        prompt.append(i).append(" --- ").append(c.getName()
         ).append('\n');
        ++i;
    }
    prompt.append("0 -- exit\n");
    int input = -1;
    while (input == -1){
        input = page.acceptIntUntil(prompt.toString(), i-1);
        if ( !includeArchived && input != 0 && campaigns
         .get(input - 1).getArchived()){
            page.nextScreen();
            page.print("Please choose something not
             actively archived");
        } else break;
    }


    page.nextScreen();
    if (input == 0) return null;
    else return campaigns.get(input-1);
}

/**
```

- `updateCampaign` replaces params

```java
public static void updateCampaign(User user){
    Campaign c = chooseCampaign(user);
    if (c == null){
        return;
    }
    String prompt = """
```

```java
public static void updateCampaign(List<Campaign> campaigns){
    Campaign c = chooseCampaign(campaigns);
    if (c == null){
        return;
    }
    String prompt = """
```

- `Campaign` changes
  - new `Campaign` constructor

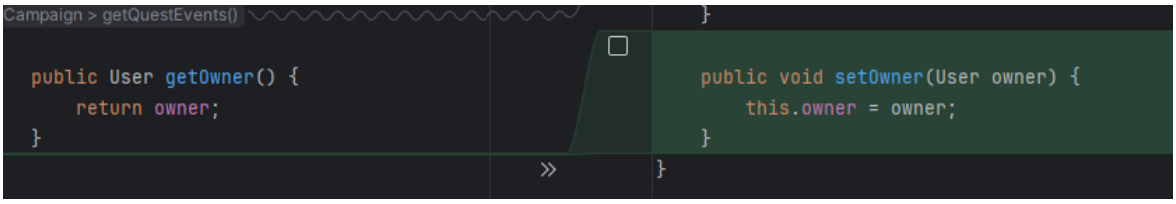
```java
 */
public Campaign(String name, User owner) {
    campaignId = UUID.randomUUID();
    this.name = name;

    this.questEvents = new ArrayList<QuestEvent>();
    this.sharedWith = new HashMap<User, PermissionLevel>
     ();
    archived=false;
}
/**
 * Add event.
 *
 * @param event the event
Campaign > addEvent()

/**
 * Remove event.
 *
 * @param eventId the event id
 */
public void removeEvent(UUID otherId){
    questEvents.removeIf( QuestEvent q -> q.getEventId()
     == otherId);
}
Campaign > getQuestEvents()
```

```java
 */
public Campaign(String name, User owner) {
    campaignId = UUID.randomUUID();
    this.name = name;

    this.questEvents = new ArrayList<QuestEvent>();
    this.sharedWith = new HashMap<User, PermissionLevel>();
    archived=false;
}

/**
 * Instantiates a new Campaign. PLEASE ASSIGN A USER AFTER
   USING THIS
 *
 * @param name the name
 */
public Campaign(String name) {
    campaignId = UUID.randomUUID();
    this.name = name;
    this.owner = null; // MAKE SURE TO SET AFTERWARDS
    this.visibility = VisibilityType.PRIVATE;
    this.questEvents = new ArrayList<QuestEvent>();
    this.sharedWith = new HashMap<User, PermissionLevel>();
    archived=false;
}
```

  - created a `setOwner` setter

- 

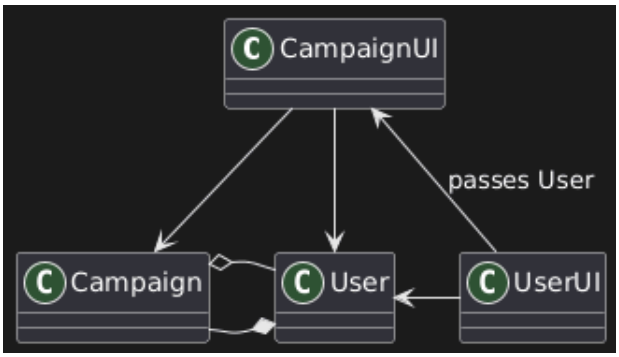
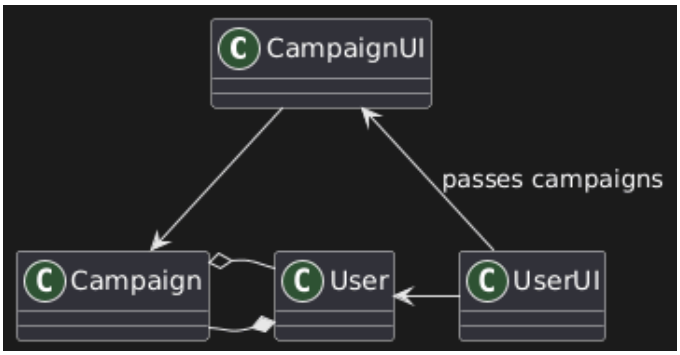
Motivation / Rationale

> explanation that is sufficiently clear and descriptive, balancing thoroughness and brevity. Demonstrates a good understanding of the refactoring and its principles?

`CampaignUI` had several methods that accepted a `User` as a parameter only to use its `.getCampaigns()` method. This passing of `User` instead of just the campaigns associated with the user let the `CampaignUI` class see too much information.

Impact

> How far-reaching is the change to incorporate the pattern, how much it changed the design.

Before Refactor	After Refactor
	

We removed a dependency from CampaignUI to User. We had to add another constructor to Campaign, and modify the parameters in CampaignUI.
Overall, the change made the design a little simpler and more intuitive. Lower coupling and separation of concerns, as well as following the Law of Demeter more closely.

2 - Unifying Duplicated Code

The issue was this: Duplicated Code
The refactor was this: Extract Method

Implementation

- `CharacterUI` changes:
  - Refactored `chooseCharacter(User)` and `chooseCharacter(QuestEvent)` to `chooseCharacter(List<QuestEvent>)`. Removed the other previous 2 methods.

- ```
  public static Character chooseCharacter(User user){
      StringBuilder prompt = new StringBuilder("Please choose a Character\n");
      Campaign choice = null;
      int i = 1;
      if (user.getCharacters().isEmpty()){
          page.print("Sorry, there are no Characters, please choose something else\n");
      }
      for (Character ch : user.getCharacters()){
          prompt.append(i).append(" --- ").append(ch.getName()).append('\n');
          ++i;
      }
      prompt.append("0 -- exit\n");
      int input = -1;
      input = page.acceptIntUntil(prompt.toString(), i-1);

      page.nextScreen();
      if (input == 0) return null;
      else return user.getCharacters().get(input-1);
  }
  public static Character chooseCharacter(QuestEvent qe){
      StringBuilder prompt = new StringBuilder("Please choose a Character\n");
      Campaign choice = null;
      int i = 1;
      if (qe.getParticipants().isEmpty()){
          page.print("Sorry, there are no characters in this quest, please choose something else\n");
      }
  ```
  ```
  public static Character chooseCharacter(List<Character> characters){
      StringBuilder prompt = new StringBuilder("Please choose a Character\n");
      Campaign choice = null;
      int i = 1;
      if (characters.isEmpty()){
          page.print("Sorry, there are no Characters, please choose something else\n");
      }
      for (Character ch : characters){
          prompt.append(i).append(" --- ").append(ch.getName()).append('\n');
          ++i;
      }
      prompt.append("0 -- exit\n");
      int input = -1;
      input = page.acceptIntUntil(prompt.toString(), i-1);

      page.nextScreen();
      if (input == 0) return null;
      else return characters.get(input-1);
  }
  }
  ```
- updated `removeCharacter` and `editCharacter` methods to use `user.getCharacters()` for parameter instead of just `user`.
  - ```
    }
    public static void removeCharacter(User user){
        Character character = chooseCharacter(user);
        if (character == null) return;
        user.getCharacters().remove(character);
    ```
    ```
    }
    public static void removeCharacter(User user){
        Character character = chooseCharacter(user
         .getCharacters());
        if (character == null) return;
        user.getCharacters().remove(character);
    ```
  - ```
    }
    private static void editCharacter(User user){
        Character character = chooseCharacter(user);
        if (character == null) {
            return;
        }
    ```
    ```
    }
    private static void editCharacter(User user){
        Character character = chooseCharacter(user,
         .getCharacters());
        if (character == null) {
            return;
    ```
- `QuestEventUI` changes:
  - Edited `removeCharacter` and `addCharacter` to use new method, passing in c.getOwner().getCharacters() instead of just c.getOwner();
    - ```
      QuestEventUI > addCharacter()
      int choice = page.acceptIntUntil("1 --- Add
       existing character\n2 --- add new character\n0
       --- go back\n", 2);
      if (choice ==0) return;
      Character character = null;
      if (choice == 1) {
          character = CharacterUI.chooseCharacter(c,
           .getOwner());
      } else if (choice == 2){
          character = CharacterUI.createCharacter(c
           .getOwner());
      ```
      ```
      QuestEventUI > addCharacter()
      int choice = page.acceptIntUntil("1 --- Add
       existing character\n2 --- add new character\n0
       --- go back\n", 2);
      if (choice ==0) return;
      Character character = null;
      if (choice == 1) {
          character = CharacterUI.chooseCharacter(c
           .getOwner().getCharacters());
      } else if (choice == 2){
          character = CharacterUI.createCharacter(c
           .getOwner());
      ```
    - ```
      private static void removeCharacters(QuestEvent qe,
       Campaign c){
          qe.getParticipants().remove(CharacterUI
           .chooseCharacter(c.getOwner()));
      }
      ```
      ```
      private static void removeCharacters(QuestEvent qe,
       Campaign c){
          qe.getParticipants().remove(CharacterUI
           .chooseCharacter(c.getOwner().getCharacters()));
      }
      ```
  - Edited `punishCharacter` and `rewardCharacter`, to use the new method. Now pass `qe.getParticipants()` instead of just `qe.
      ```

```java
private static void rewardCharacter(QuestEvent qe,
 Campaign c){
    if (qe.getParticipants().isEmpty()){
        page.print("No characters to reward.\n");
        return;
    }
    Character character = CharacterUI
     .chooseCharacter(qe); // only questEvent
     characters
    if (character == null) return;
    InventoryUI.addItem(character);
}
private static void punishCharacter(QuestEvent qe){
    if (qe.getParticipants().isEmpty()){
        page.print("No characters to punish.\n");
        return;
    }
    Character character = CharacterUI
     .chooseCharacter(qe); // only questEvent
     characters
    if (character == null) return;
    InventoryUI.removeItem(character);
}
```

```java
private static void rewardCharacter(QuestEvent qe,
 Campaign c){
    if (qe.getParticipants().isEmpty()){
        page.print("No characters to reward.\n");
        return;
    }
    Character character = CharacterUI
     .chooseCharacter(qe.getParticipants()); // only
      questEvent characters
    if (character == null) return;
    InventoryUI.addItem(character);
}
private static void punishCharacter(QuestEvent qe){
    if (qe.getParticipants().isEmpty()){
        page.print("No characters to punish.\n");
        return;
    }
    Character character = CharacterUI
     .chooseCharacter(qe.getParticipants()); // only
      questEvent characters
    if (character == null) return;
    InventoryUI.removeItem(character);
}
```

Motivation / Rationale

> explanation that is sufficiently clear and descriptive, balancing thoroughness and brevity. Demonstrates a good understanding of the refactoring and its principles?

Within my CharacterUI class, I had a method with the same name with 2 different parameters. A `chooseCharacter(User)` and a `chooseCharacter(QuestEvent)`. These did the same operation, they only differed in the characters that they decided to choose *from*. because of this, I figured that an easy refactor would be to extract the main algorithm and have them pass only the characters that they want to choose from. The benefits of this were: removing some 20 lines of code, simplifying / generalizing the method, and reducing the error space. The method I extracted was `chooseCharacter(List<Character>)`

Impact

> How far-reaching is the change to incorporate the pattern, how much it changed the design.

This refactor changed only 2 classes: the CharacterUI class and the QuestEventUI class, as those were the classes which had calls to the `chooseCharacter` method variations. The changes were simple 1-line parameter pass changes (instead of passing in `User user`, pass in `List<Character>` `user.getCharacters()`).
UML-wise, we simplified the `CharacterUI` class by removing 1 method.

It might have caused a message chain to appear in `QuestEventUI`, as the `c.getOwner()` parameter is now `c.getOwner().getCharacters()`. This could be circumvented by giving each campaign a method that returns this value instead. Then, this could have the `Campaign` class delegate / hide the call with a `c.getCharacters()`. This is minimal, however, so I will omit this change and simply note it here.

AI Section

> **Provide your reasoning** as to what makes the code smells, refactoring, and design patterns "the best" for your design and implementation.

1 Code Smell and Refactor

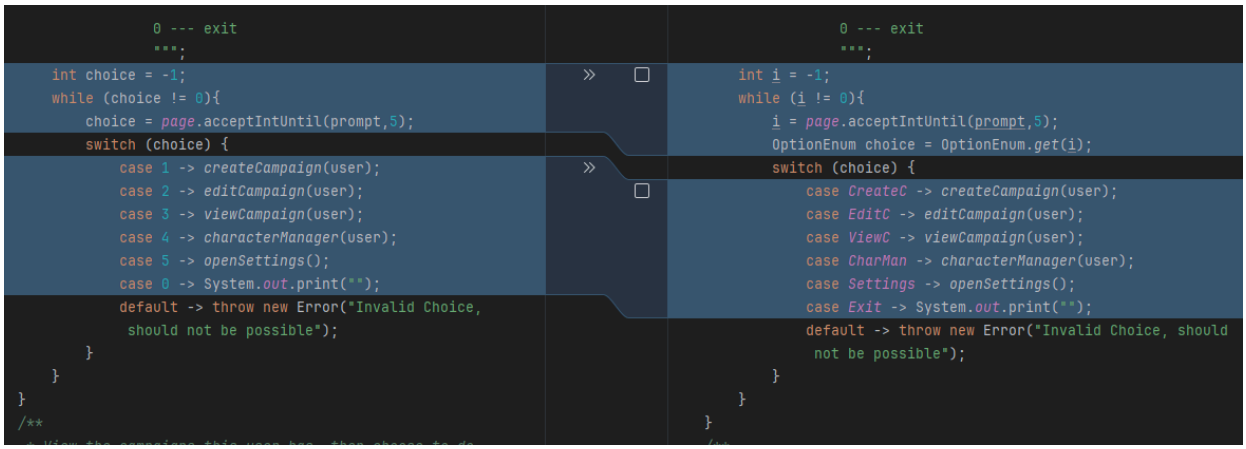
code smell: Magic Numbers and primitive obsession
Refactor: Replace Magic Number with Symbolic Constant

Motivation / Rationale

- It recommended getting rid of the hard-coded magic numbers I had for switch statements here:
  - ```
    switch (choice) {
        case 1 -> createCampaign(user);
        case 2 -> editCampaign(user);
        case 3 -> viewCampaign(user);
        case 4 -> characterManager(user);
        case 5 -> openSettings();
        case 0 -> System.out.print("");
        default -> throw new Error("Invalid Choice, should not be possible");
    ```
  - I appreciated this recommendation as it increases readability in a satisfying enough manner, without introducing much complexity. It recommended an Enum to replace these numbers which, although the professor is not fond of them, I found simple enough for this iteration. Formally, it could be considered "Replace Data Value with Object". I also found that this helped a little bit against primitive obsession.
  - This was the best and easiest code smell that I could have addressed in the time that I had left. It was an actual smell that makes my code more readable.

Implementation

- Created `OptionEnum` enum
  - ```
    public class UserUI {  4 usages  Gilberto Froylan Fernandez *
        private enum OptionEnum{  4 usages  new *
            Exit( i: 0), CreateC( i: 1), EditC( i: 2), ViewC( i: 3), CharMan( i: 4), Settings( i: 5);  1 usage
            OptionEnum(int i) {}
            static OptionEnum get(int i) { return OptionEnum.values()[i]; }
        }
    ```
- Changed switch within `displayOptions` to use the enum instead of ints.

- 

```
                0 --- exit
                """;
    int choice = -1;
    while (choice != 0){
        choice = page.acceptIntUntil(prompt,5);
        switch (choice) {
            case 1 -> createCampaign(user);
            case 2 -> editCampaign(user);
            case 3 -> viewCampaign(user);
            case 4 -> characterManager(user);
            case 5 -> openSettings();
            case 0 -> System.out.print("");
            default -> throw new Error("Invalid Choice,
             should not be possible");
        }
    }
}
/**
```

```
                0 --- exit
                """;
    int i = -1;
    while (i != 0){
        i = page.acceptIntUntil(prompt,5);
        OptionEnum choice = OptionEnum.get(i);
        switch (choice) {
            case CreateC -> createCampaign(user);
            case EditC -> editCampaign(user);
            case ViewC -> viewCampaign(user);
            case CharMan -> characterManager(user);
            case Settings -> openSettings();
            case Exit -> System.out.print("");
            default -> throw new Error("Invalid Choice, should
             not be possible");
        }
    }
}
```

Impact

- We created an additional private enum within `UserUI` and modified `displayOptions`. The switch is now more readable and straightforward with minimal changes
- I thought it would conflict with the Command pattern set up in the `InventoryUI` class, but that class still has to map arbitrary (magic) numbers into the specific commands, so it may still be applicable there.

Replicability and Evidence

> Are the prompts, responses, modifications, and associated reasoning behind modifications provided?

See "CodeSmellRefactorAIPrompt.txt"
See "UserUIDotJava.txt" as input

I ignored most of the other code it gave me, and decided to focus only on the magic numbers.
I used ZotGPT, I have attached the .txt files and omitted nothing.

1 Design Pattern

Pattern: State Pattern

Motivation / Rationale

Campaign has 2 distinct states, archived and not archived. An archived campaign should not be able to do certain operations, but it is technically possible given the current implementation, and it would be better to separate those concerns. This way, I can keep the same operations but just perform different actions based on the state of the Campaign.
This was one of the "best" for my design and implementation because it does create more security around which kinds of Campaigns can be edited and which ones cannot. Having this information segregated also helps narrow down the error-space for the future.

Implementation

- Changes to `CampaignUI`
  - Replaced action methods with a delegate to the specific `CampaignState` implementing objects, so that they can perform the required actions.
    - The example below is for the `updateName` method, but the changes were identical for the methods listed in the `CampaignState` interface below.
    - ```
      private static void updateName(Campaign c){  1 usage
          CampaignState cState = c.getState();
          cState.updateName();
      }
      ```
- Changes to `Campaign`
  - added `state` var initialized to `ActiveCampaign` in constructor
    - ```
      private CampaignState state;
      ```
    - ```
      state = new ActiveCampaign(this);
      ```
  - modified `setArchived` to update State
    - ```
      public void setArchived(boolean archived) {
          this.archived = archived;
          if (archived){
              state = new ArchivedCampaign(this);
          } else state = new ActiveCampaign(this);
      }
      ```
  - created `getState`
    - ```
      public CampaignState getState(){return state;};
      ```
- Created `CampaignState` interface
  - Wrote action methods from CampaignUI class.
    - ```
      package Core;

      public interface CampaignState {
          void archive();  1 usage  2 imple
          void changeVisibility();  1 us
          void updateName();  1 usage  2 i
          void addQuestEvent();  1 usage
          void updateQuestEvent();  1 us
          void removeQuestEvent();  1 us
          void viewCampaign();  1 usage
      }
      ```
- created `ActiveCampaign` class
  - initializes a `Campaign` instance in a field;

```java
public class ActiveCampaign implements CampaignState{
    Campaign campaign;
    private static final Page page = Page.getPage();

    public ActiveCampaign(Campaign c) {
        campaign = c;
    }
```

- Implemented all `CampaignUI` action methods
  - This was copy-pasted from the original `CampaignUI` code. There are more methods but they are the same. They are listed in the `CampaignState` methods.

```java
public void updateName(){
    String newName = page.acceptStrWithValidation("Please choose a new name: ");
    if (newName != null)  campaign.setName(newName);
    page.print("new name set\n");
};
public void addQuestEvent(){
    QuestEvent qe = QuestEventUI.createQuestEvent(campaign);
    if (qe != null) campaign.addEvent(qe);
};
public void updateQuestEvent(){
    page.nextScreen();
    QuestEventUI.updateQuestEvent(campaign);
};
public void removeQuestEvent(){
    page.nextScreen();
    QuestEventUI.removeQuestEvent(campaign);
};
```

- created `ArchivedCampaign` class
  - constructor that initializes a `Campaign` instance in a field

```java
public class ArchivedCampaign implements CampaignState{
    Campaign campaign;
    private static final Page page = Page.getPage();

    public ArchivedCampaign(Campaign c) {
        campaign = c;
    }
```

  - Created custom error code for this page.

```java
private void printArchiveError(){
    page.print("Sorry, you cannot modify an archived page. Please choose another option.\n");
}
```

  - Blocked certain method calls.

```
public void changeVisibility(){printArchiveError();};
public void updateName(){printArchiveError();};
public void addQuestEvent(){printArchiveError();};
public void updateQuestEvent(){printArchiveError();};
public void removeQuestEvent(){printArchiveError();};
```

- Implemented view method.

```
public void viewCampaign(){  1 usage
    String display = """
            Name: %s
            Owner: %s
            Visibility: %s
            QuestEvents:
            %s
            """.formatted(campaign.getName(),
            campaign.getOwner().getUsername(),
            campaign.getVisibility().toString(),
            QuestEventUI.timelineString(campaign)
    );
    page.print(display);
}
```

Impact

We added 2 more classes and 1 more interface, as well as added a new field (and dependency) into the `Campaign` class to hold state.
Dependencies were removed from the `CampaignUI` class.
There is a lot of duplicated code within the CampaignUI class as it now follows the same predictable format of : `CampaignState cState = c.getState(); cState.__method_to_run__();` . This could be further refactored in future iterations.

Replicability and Evidence

> Are the prompts, responses, modifications, and associated reasoning behind modifications provided?

See "DesignPatternAIPrompt.txt"
See "CampaignUIDotJava.txt" as input

Most of the code I created were the classes and I had to copy-paste the methods from `CampaignUI` into the `CampaignState` implementing classes. Most of it was just moving code around classes and creating the delegates within `CampaignUI`. I had to do some minor modifications to `Campaign` class and implement some specific archive methods, but aside from that nothing crazy. I did decide that instead of changing the existing `Campaign` class code and using that as a delegate, I decided that I'd rather not move around the backend logic too much and shift towards moving the `CampaignUI` methods and using this class as a delegate, as it seemed more sanitary to change the UI code rather

than the backend code.
I used ZotGPT, I have attached the .txt files and omitted nothing.